Relatório: Configuração Centralizada com Spring Cloud Config Server

1. O Fluxo de Inicialização e Obtenção das Configurações

Neste trabalho, construímos um sistema distribuído composto por um Servidor de Configuração e três aplicações clientes (Vendas, Estoque e Relatórios). A ideia principal é que os clientes não guardem suas próprias configurações localmente, mas sim busquem tudo de um repositório remoto no GitHub.

O fluxo de funcionamento acontece numa ordem estrita:

1. **O Servidor:** Primeiro, iniciamos o Config Server. Ao rodar, ele se conecta ao nosso repositório no GitHub para ler os arquivos de configuração que estão lá. Ele fica escutando na porta 8888.
2. **Os Clientes:** Depois, iniciamos os clientes. Durante a sua inicialização, antes mesmo de terminarem de subir o Tomcat (servidor web interno), eles pausam e fazem uma requisição HTTP para o Config Server na porta 8888.
3. **A Resposta:** O cliente avisa seu nome (ex: `cliente-vendas`) e seu perfil (ex: `dev`). O servidor junta as duas coisas, procura no GitHub o arquivo `cliente-vendas-dev.properties`, transforma o conteúdo em JSON e devolve para o cliente. Com os dados em mãos, o cliente termina de iniciar na porta correta (ex: 8081) e com a mensagem certa.

2. A Função de cada Propriedade no application.properties

Os arquivos `application.properties` locais são bem curtos, servindo apenas como um "mapa" inicial.

No Servidor:

- `spring.application.name=config-server`: Dá o nome à aplicação.
- `server.port=8888`: Fixa a porta do servidor, que é o padrão do Spring Cloud.
- `spring.cloud.config.server.git.uri`: Aponta a URL do nosso repositório no GitHub onde os arquivos reais estão guardados.
- `spring.cloud.config.server.git.clone-on-start=true`: Força o servidor a clonar o repositório logo que liga, agilizando as respostas.
- `management.endpoints.web.exposure.include=*`: Libera as URLs de monitoramento do Actuator (como o /health).

Nos Clientes (exemplo do cliente de vendas):

- `spring.application.name=cliente-vendas`: É a chave de tudo. É esse nome que o servidor usa para procurar o arquivo lá no GitHub.
- `spring.profiles.active=dev`: Diz ao servidor qual versão do arquivo ele quer (juntando com o nome, ele procura `cliente-vendas-dev.properties`).
- `spring.config.import=optional:configserver:http://localhost:8888`: É o endereço do Config Server. É aqui que o cliente sabe onde bater para buscar seus dados.

3. Anotações (Tags) Utilizadas no Código

Para fazer a mágica acontecer com pouco código, usamos algumas anotações importantes do Spring Boot:

- **@EnableConfigServer:** Usada na classe principal do servidor. Transforma um projeto Spring comum em um servidor capaz de gerenciar e distribuir configurações.
- **@RestController e @GetMapping:** Usadas nos clientes para criar o endpoint `/mensagem`, permitindo testarmos no navegador se a configuração chegou certo.
- **@Value:** Usada no controlador do cliente. É ela que "suga" o valor da variável (ex: `mensagem.boasvindas`) que veio lá do GitHub e injeta direto no código Java. Caso o servidor esteja fora do ar, ela tem um texto padrão de segurança ("Configuração não carregada").
- **@RefreshScope:** Adicionada no controlador do cliente. Ela diz ao Spring: *"Fique atento, essas configurações podem mudar"*. Sem ela, o cliente lê a configuração uma vez e guarda na memória para sempre. Com ela, podemos forçar o cliente a atualizar o texto sem precisar desligar a aplicação.

4. Testes Realizados

Abaixo estão as evidências de que o sistema se conectou corretamente e as rotas estão funcionando.

A) Teste de Saúde do Servidor Ao acessar `http://localhost:8888/actuator/health`, o servidor retorna que está "UP" (funcionando).

B) Teste do Servidor Lendo o GitHub Ao acessar `http://localhost:8888/cliente-vendas/dev`, o servidor exibe o JSON provando que leu o arquivo do GitHub corretamente.

C) Teste dos Clientes (Portas 8081, 8082, 8083) Ao acessar a rota `/mensagem` de cada cliente, eles exibem o texto injetado do repositório remoto.

D) Teste de Atualização Dinâmica (Refresh) Fizemos a alteração de um arquivo no GitHub. Em seguida, no Postman, enviamos uma requisição **POST** (vazia) para `http://localhost:8081/actuator/refresh`. O Postman retornou 200 OK com um array mostrando a variável alterada. Ao atualizar a página web, o texto mudou em tempo real, comprovando a eficácia do `@RefreshScope`.